```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn import metrics
```

```
df=pd.read_csv('USA_Housing.csv')
df.head()
```

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price	Address
0	79545.46	5.68	7.01	4.09	23086.80	1059033.56	208 Michael Ferry Apt. 674\nLaurabury, NE 3701...
1	79248.64	6.00	6.73	3.09	40173.07	1505890.91	188 Johnson Views Suite 079\nLake Kathleen, CA...
2	61287.07	5.87	8.51	5.13	36882.16	1058987.99	9127 Elizabeth Stravenue\nDanielltown, WI 06482...
3	63345.24	7.19	5.59	3.26	34310.24	1260616.81	USS Barnett\nFPO AP 44820

Next steps: Generate code with df | New interactive sheet

```
df.describe()
```

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price
count	5000.00	5000.00	5000.00	5000.00	5000.00	5000.00
mean	68583.11	5.98	6.99	3.98	36163.52	1232072.65
std	10657.99	0.99	1.01	1.23	9925.65	353117.63
min	17796.63	2.64	3.24	2.00	172.61	15938.66
25%	61480.56	5.32	6.30	3.14	29403.93	997577.14
50%	68804.29	5.97	7.00	4.05	36199.41	1232669.38
75%	75783.34	6.65	7.67	4.49	42861.29	1471210.20

```
df.isnull().sum()
```

	0
Avg. Area Income	0
Avg. Area House Age	0
Avg. Area Number of Rooms	0
Avg. Area Number of Bedrooms	0
Area Population	0
Price	0
Address	0

dtype: int64

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
if 'Address' in df.columns:
  df.drop(['Address'],axis=1,inplace=True)
```

```
df.head()
```

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price
0	79545.46	5.68	7.01	4.09	23086.80	1059033.56
1	79248.64	6.00	6.73	3.09	40173.07	1505890.91
2	61287.07	5.87	8.51	5.13	36882.16	1058987.99
3	63345.24	7.19	5.59	3.26	34310.24	1260616.81

Next steps: Generate code with df | New interactive sheet

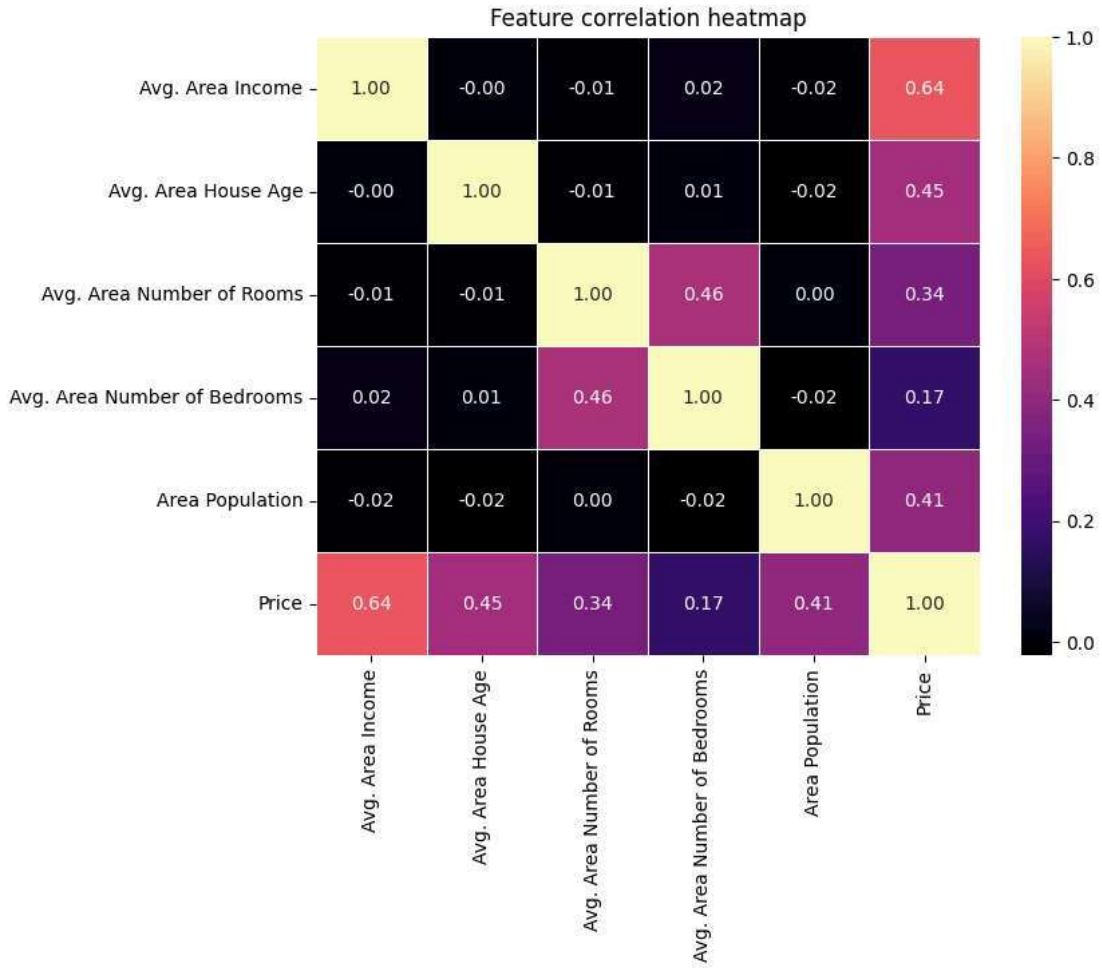
```
pd.set_option('display.float_format','{:.2f}'.format)
```

```
df.head()
```

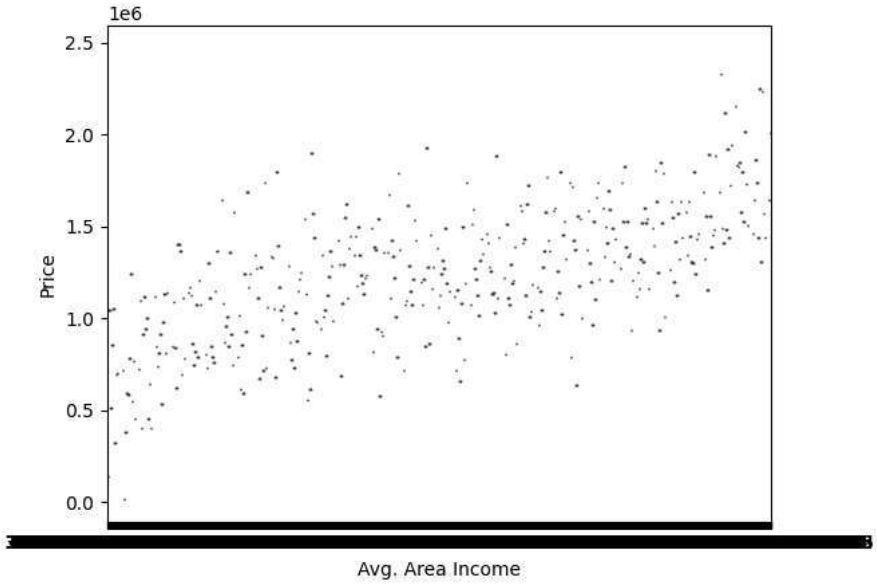
	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price
0	79545.46	5.68	7.01	4.09	23086.80	1059033.56
1	79248.64	6.00	6.73	3.09	40173.07	1505890.91
2	61287.07	5.87	8.51	5.13	36882.16	1058987.99
3	63345.24	7.19	5.59	3.26	34310.24	1260616.81

Next steps: Generate code with df | New interactive sheet

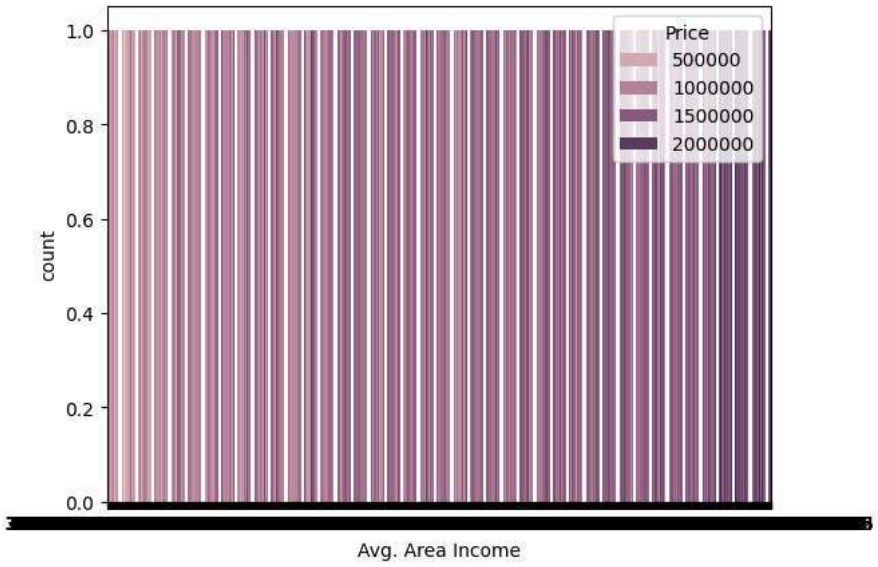
```
plt.figure(figsize=(8,6))
corr_matrix=df.corr()
sns.heatmap(corr_matrix,cmap='magma',annot=True,fmt=".2f",linewidth=0.5)
plt.title("Feature correlation heatmap")
plt.show()
```



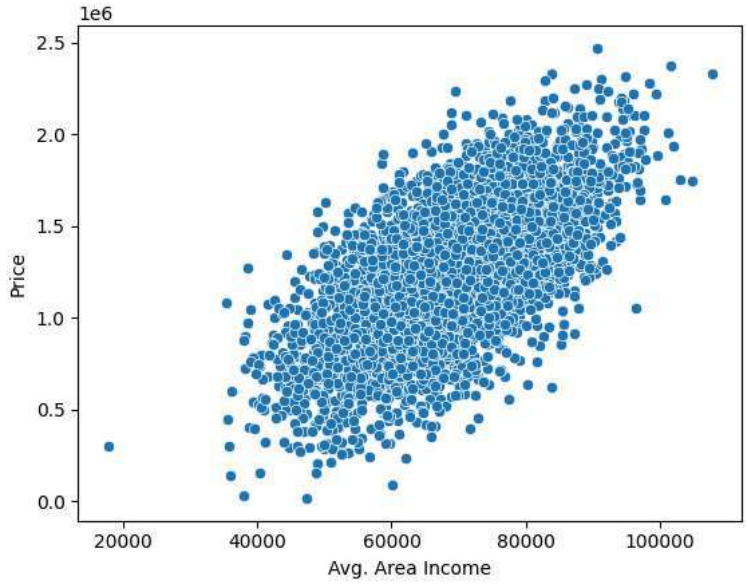

```
sns.boxplot(x=df["Avg. Area Income"],y=df["Price"])
plt.show()
```




```
sns.countplot(x="Avg. Area Income",hue="Price", data=df)
plt.show()
```




```
sns.scatterplot(x="Avg. Area Income",y="Price",data=df)
plt.show()
```



Simple Linear Regression

```
x=df[['Avg. Area Income']]
y=df['Price']
```

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

```
from sklearn.linear_model import LinearRegression
slr_model=LinearRegression()
slr_model.fit(x_train,y_train)
```

```
▾ LinearRegression
LinearRegression()
```

```
y_pred=slr_model.predict(x_test)
```

```
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
```

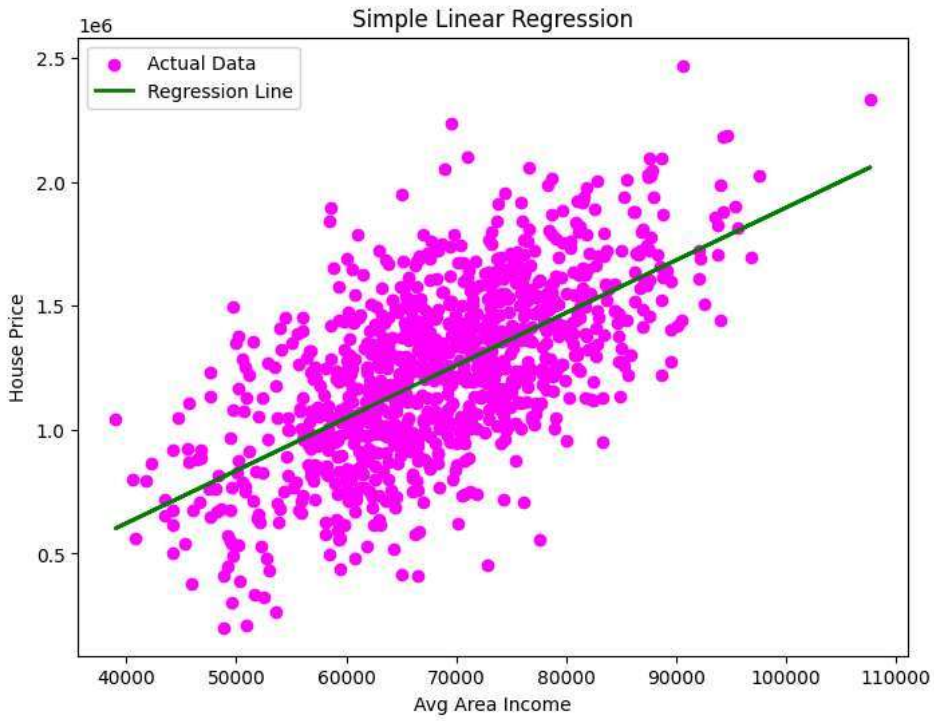
```
from sklearn.metrics import r2_score
print("Simple linear regression results")
print("Mean absolute error: ",mean_absolute_error(y_test,y_pred))
print("Mean squared error: ",mean_squared_error(y_test,y_pred))
print("RMSE:",np.sqrt(mean_squared_error(y_test,y_pred)))
print("R2 score: ",r2_score(y_test,y_pred))
```

```
Simple linear regression results
Mean absolute error:  216826.35309989064
Mean squared error:  74194887058.66
RMSE: 272387.38417676394
R2 score:  0.39694865192619766
```

```
print("Intercept:",slr_model.intercept_)
print("Slope:",slr_model.coef_)
```

```
Intercept: -225053.70287170145
Slope: [21.20890629]
```

```
plt.figure(figsize=(8,6))
plt.scatter(x_test,y_test,color='magenta',label='Actual Data')
plt.plot(x_test,y_pred,color='green',linewidth=2,label='Regression Line')
plt.xlabel("Avg Area Income")
plt.ylabel("House Price")
plt.title("Simple Linear Regression")
plt.legend()
plt.show()
```




```
income=float(input("Enter avg area income:"))
new_data=np.array([[income]])
prediction=slr_model.predict(new_data)
print("Predicted house price=${:,.2f}".format(prediction[0]))
```

```
Enter avg area income:900000
Predicted house price=$18,862,961.96
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
  warnings.warn(
```

Hyperparameter Tuning

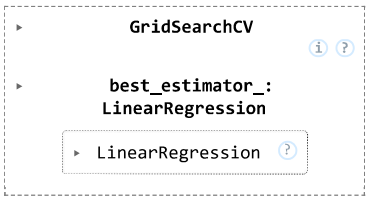
```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import mean_absolute_error,mean_squared_error,r2_score
import numpy as np
```

```
lr=LinearRegression()
```

```
lr_params={
    "fit_intercept":[True,False],
    "positive":[True,False],
}
```

```
grid_search=GridSearchCV(
    estimator=lr,
    param_grid=lr_params,
    cv=5,
    scoring='r2',
    n_jobs=-1
    )
```

```
grid_search.fit(x_train,y_train)
```




```
print("Best parameters:",grid_search.best_params_)
print(grid_search.best_params_)
```

```
Best parameters: {'fit_intercept': True, 'positive': True}
{'fit_intercept': True, 'positive': True}
```

```
{'fit_intercept':True,
 'positive':False}
```

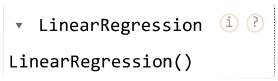
```
{'fit_intercept': True, 'positive': False}
```

```
print("Best cross validation score:")
print(grid_search.best_score_)
```

```
Best cross validation score:
0.4111086193361001
```

```
best_model=grid_search.best_estimator_
```

```
LinearRegression(
    fit_intercept=True,
    positive=False
)
```




```
y_pred=best_model.predict(x_test)
```

```
mae=mean_absolute_error(y_test,y_pred)
mse=mean_squared_error(y_test,y_pred)
rmse=np.sqrt(mse)
r2_score=r2_score(y_test,y_pred)
print("Mean absolute error:",mae)
print("Mean squared error:",mse)
print("Root mean squared error:",rmse)
print("R2 score:",r2_score)
```

```
Mean absolute error: 216826.35309989064
Mean squared error: 74194887058.66
Root mean squared error: 272387.38417676394
R2 score: 0.39694865192619766
```

```
print(best_model)
```

```
LinearRegression(positive=True)
```

```
import numpy as np

income = float(input("Enter avg area income: "))

test_input = np.array([[income]])

predicted_price = slr_model.predict(test_input)
print("Predicted house price = ${:,.2f}".format(predicted_price[0]))
```

```
Enter avg area income: 90000
Predicted house price = $1,683,747.86
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
  warnings.warn(
```

Ridge Regression

```
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import r2_score
```

```
from sklearn.linear_model import Ridge
ridge = Ridge()
ridge.fit(x_train, y_train)
ridge_pred = ridge.predict(x_test)

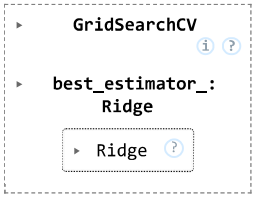
print("Default Ridge r2:", r2_score(y_test, ridge_pred))
```

```
Default Ridge r2: 0.3969486519262033
```

```
param_grid={'alpha':[0.001,0.01,0.1,1,10,100]}
```

```
grid_ridge=GridSearchCV(estimator=Ridge(),param_grid=param_grid,
                        scoring='r2',
                        cv=5)
```

```
grid_ridge.fit(x_train,y_train)
```




```
print("Best Alpha:",grid_ridge.best_params_)
```

```
Best Alpha: {'alpha': 100}
```

```
print(grid_ridge.best_score_)
```

```
0.41110861933657433
```

```
best_ridge=grid_ridge.best_estimator_
ridge_pred=best_ridge.predict(x_test)
```

```
print("MAE:",mean_absolute_error(y_test,ridge_pred))
print("MSE:",mean_squared_error(y_test,ridge_pred))
print("RMSE:",np.sqrt(mean_squared_error(y_test,ridge_pred)))
print("R2 score:",r2_score(y_test,ridge_pred))
```

```
MAE: 216826.35310007748
MSE: 74194887058.58911
RMSE: 272387.38417663384
R2 score: 0.39694865192677387
```

LASSO REGRESSION

```
from sklearn.linear_model import Lasso
param_grid={'alpha':[0.001,0.01,0.1,1,10]}
```

```
grid_lasso=GridSearchCV(Lasso(max_iter=5000),param_grid,
                        cv=5,scoring='r2')
grid_lasso.fit(x_train,y_train)
```

GridSearchCV

best_estimator_: Lasso

Lasso

```
print(grid_lasso.best_params_)
```

```
{'alpha': 10}
```

```
lasso_pred=grid_lasso.predict(x_test)
print(r2_score(y_test,lasso_pred))
```

```
0.39694865193706386
```

```
predictions=slr_model.predict(x_test)
```

```
plt.figure(figsize=(8,6))
sns.scatterplot(x=y_test, y=predictions,color='pink',label='Predictions')
plt.plot([min(y_test),max(y_test)],[min(y_test),max(y_test)],color='red',linewidth=2,label='Perfect Fit')
plt.title("Actual vs Predicted Values")
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.legend()
plt.show()
```

